

Module 3: Computability, Undecidability & System Limits

CS 305: Theory of Computation (Weeks 9–12)

Department of Computer Science

Ghana Institute of Management and Public Administration (GIMPA)

Level 300 Undergraduate

Module 3 Roadmap: Weeks 9 to 12

Theoretical Foundations

- **Wk 9:** Turing Machines (7-tuple) & TM Variants
- **Wk 10:** The Church–Turing Thesis & Algorithms
- **Wk 11:** Decidability of Regular/CFL Problems
- **Wk 12:** Halting Problem, Diagonalization & Rice's Theorem

Research Literature & System Spotlights

- **Wk 9: ICLR 2024 / ICML 2023:** Chain-of-Thought LLMs as Universal Turing Machines
- **Wk 10:** Universal Computation & Physical Bounds
- **Wk 11: POPL / SLAM:** Static Code Verification
- **Wk 12: ACM CCS 2016:** Ethereum Gas Mechanics

Week 9: Formal Definition of a Turing Machine

Formal 7-Tuple Definition (Sipser Def 3.3)

A **Turing Machine (TM)** is a 7-tuple $(Q, \Sigma, \Gamma, \delta, q_0, q_{\text{accept}}, q_{\text{reject}})$:

- 1 Q is the finite set of states.
- 2 Σ is the input alphabet (not containing blank symbol).
- 3 Γ is the tape alphabet ($\Sigma \subset \Gamma$ and $\epsilon \in \Gamma$).
- 4 $\delta : Q \times \Gamma \rightarrow Q \times \Gamma \times \{L, R\}$ is the transition function.
- 5 $q_0 \in Q$ is the start state.
- 6 $q_{\text{accept}} \in Q$ is the accept state.
- 7 $q_{\text{reject}} \in Q$ is the reject state ($q_{\text{accept}} \neq q_{\text{reject}}$).

Read/Write Head (State q)



Seminal Reading: ICLR 2024 / ICML 2023

Merrill & Sabharwal (Allen AI / Univ. of Washington), "*The Expressive Power of Transformers with Chain of Thought*," ICLR 2024; **Schuermans et al. (Google Brain)**, ICML 2023.

Transformer Without CoT (Direct Output)

- Forward pass has fixed circuit depth L .
- Cannot iterate or execute unbounded dynamic steps.
- Strictly bounded in circuit complexity (TC^0).

Transformer + Chain-of-Thought (Scratchpad)

- Output tokens serve as an **infinite Turing read/write tape**.
- The model stores state, updates memory, and loops.
- **Proved Turing-Complete (Universal Computation)!**

Theorem: Equivalence of Turing Machine Variants

The following models all recognize the **exact same class of languages**:

- Single-tape Deterministic Turing Machines
- Multi-tape Turing Machines (reading k tapes simultaneously)
- Nondeterministic Turing Machines (NTMs)
- Random-Access Memory (RAM) computers, C, Python, Java, Lambda Calculus

The Church–Turing Thesis

Intuitive notion of an algorithm $\equiv$ Turing Machine algorithm

Any algorithmic calculation that can be physically executed in the universe can be computed by a Turing Machine.

Week 11: Decidable vs. Recognizable Languages

Turing-Recognizable (Recursively Enumerable)

A machine M recognizes L if for any string $w \in L$, M halts and accepts. If $w \notin L$, M may reject or **loop forever**.

Turing-Decidable (Recursive)

A machine M **decides** L if M halts on **every** input string (guaranteed never to loop).

$$M(w) = \begin{cases} \text{Accept} & \text{if } w \in L \\ \text{Reject} & \text{if } w \notin L \end{cases}$$

Decidable Problems Concerning Formal Languages (Sipser Ch. 4.1)

- $A_{\text{DFA}} = \{\langle B, w \rangle \mid B \text{ is a DFA that accepts } w\}$ is **decidable**.
- $E_{\text{DFA}} = \{\langle B \rangle \mid B \text{ is a DFA and } L(B) = \emptyset\}$ is **decidable** (BFS graph reachability).
- $EQ_{\text{DFA}} = \{\langle A, B \rangle \mid A, B \text{ are DFAs and } L(A) = L(B)\}$ is **decidable** (symmetric difference).

Week 12: The Halting Problem & Cantor's Diagonalization

Theorem (Alan Turing, 1936; Sipser Thm 4.11)

$A_{TM} = \{\langle M, w \rangle \mid M \text{ is a TM and } M \text{ accepts } w\}$ is **UNDECIDABLE!**

Proof by Diagonal Contradiction

❶ Assume decider $H(\langle M, w \rangle)$ exists.

❷ Build opposite machine $D(\langle M \rangle)$:

$$D(\langle M \rangle) = \begin{cases} \text{Reject} & \text{if } H(\langle M, \langle M \rangle \rangle) = \text{Accept} \\ \text{Accept} & \text{if } H(\langle M, \langle M \rangle \rangle) = \text{Reject} \end{cases}$$

❸ Run $D(\langle D \rangle) \implies D$ accepts iff D rejects!
Contradiction!

Rice's Theorem (Sipser Problem 5.28)

Any non-trivial semantic property of a Turing Machine's language is undecidable.

- Is $L(M)$ empty? **Undecidable!**
- Is $L(M)$ regular? **Undecidable!**
- Does M contain a virus? **Undecidable!**

Seminal Reading: ACM CCS 2016

Luu et al. (National Univ. of Singapore), "Making Smart Contracts Smarter," ACM Conference on Computer and Communications Security (CCS 2016).

Case Study 1: Ethereum Smart Contract Gas

- The EVM is Turing-complete.
- Miners cannot statically check if a contract has an infinite loop (A_{TM} undecidable).
- **Solution:** Charge **Gas** for every execution opcode. If a contract loops, it runs out of gas and halts!

Case Study 2: Anti-Virus Software Limitations

- Can an AV company build a software analyzer that 100% correctly identifies all malware?
- **No!** By Rice's Theorem (Cohen 1987), determining whether arbitrary binary code contains malware is undecidable.

Module 3 Summary & Looking Ahead to Complexity

Key Takeaways

- **Turing Machines** define the absolute boundary of what algorithms can solve.
- Some problems are fundamentally **undecidable** (Halting problem, Rice's theorem) — no amount of AI or compute power can ever solve them!

Looking Ahead: Module 4 (Computational Complexity & Class NP)

For problems that *are* decidable, can we solve them **efficiently in polynomial time**? → P vs. NP and the **Millenium Prize**!